

Lab 08: Dynamic Programming 3

Introduction to Dynamic Programming (動態規劃簡介)

Dynamic Programming (DP) is a powerful algorithmic technique used for solving complex problems by breaking them down into simpler, overlapping subproblems. The results of these subproblems are stored (usually in an array or a table) to avoid redundant computations, leading to efficient solutions. This approach is particularly effective when a problem exhibits two key characteristics:

1. Optimal Substructure (**最佳子結構**): An optimal solution to the overall problem can be constructed from the optimal solutions of its subproblems.
(問題的最佳解可由其子問題的最佳解有效地建構而成。)
2. Overlapping Subproblems (**重疊子問題**): The same subproblems are encountered and solved multiple times during the recursive computation. DP stores the solutions to these subproblems (memoization or tabulation) to reuse them.
(相同的子問題在遞迴計算過程中會被多次遇到並求解。動態規劃會儲存這些子問題的解 (使用記憶化或列表法) 以便重複使用。)

Unlike greedy algorithms, which make a locally optimal choice at each step hoping to find a global optimum, dynamic programming typically explores all possibilities by considering different choices for subproblems and then combining their results to find the best overall solution.

There are two main approaches to implementing DP:

- * Memoization (Top-Down with Memoization / **記憶化遞迴 - 由上而下**): Solve the problem recursively, but store the result of each computed subproblem in a lookup table. If a subproblem is encountered again, return the stored result instead of recomputing.
(透過遞迴方式解決問題，但將每個已計算子問題的結果儲存在查詢表中。如果再次遇到相同的子問題，則直接回傳儲存的結果，而非重新計算。)
- * Tabulation (Bottom-Up / **列表法 - 由下而上**): Solve subproblems in a specific order, typically starting from the smallest base cases and iteratively building up solutions to larger subproblems. Results are stored in a table.
(依照特定順序解決子問題，通常從最小的基礎案例開始，逐步建立較小子問題的解，並將結果儲存在表格中。)

This lab will explore a few classic problems that can be solved efficiently using dynamic programming.
(本實驗將探討一些可以使用動態規劃有效解決的經典問題。)

1. Fibonacci Sequence / Climbing Stairs (費氏數列 / 爬樓梯問題)

Problem Description (問題描述):

The Fibonacci sequence is defined as follows: $F_0 = 0$, $F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$ for $n > 1$. The task is to compute the n -th Fibonacci number. (費氏數列定義如下： $F_0 = 0$, $F_1 = 1$ ，且對於 $n > 1$ ， $F_n = F_{n-1} + F_{n-2}$ 。任務是計算第 n 個費氏數。)

This problem is structurally identical to the "climbing stairs" problem where you can climb 1 or 2 stairs at a time. F_n would represent the number of distinct ways to reach the n -th stair (if we adjust base cases, e.g., ways to reach stair 1 is 1, stair 2 is 2).

(這個問題在結構上與「爬樓梯」問題相同，每次可以爬 1 階或 2 階。 F_n 可以代表到達第 n 階樓梯的不同方法數量 (如果我們調整基礎案例，例如，到達第 1 階的方法有 1 種，第 2 階有 2 種)。)

DP Approach (動態規劃方法):

A naive recursive solution $F(n) = F(n-1) + F(n-2)$ has exponential time complexity due to recomputing the same Fibonacci numbers multiple times. DP can optimize this. (一個樸素的遞迴解法 $F(n) = F(n-1) + F(n-2)$

具有指數級的時間複雜度，因為它會多次重複計算相同的費氏數。動態規劃可以優化這個問題。)

- * State Definition (狀態定義): $dp[i]$ will store the i -th Fibonacci number, F_i . ($dp[i]$ 將儲存第 i 個費氏數 F_i 。)
- * Base Cases (基礎案例): $dp[0] = 0$ $dp[1] = 1$
- * Recurrence Relation (遞迴關係式): $dp[i] = dp[i-1] + dp[i-2]$ for $i > 1$.
- * Final Answer (最終答案): The n -th Fibonacci number is $dp[n]$. (第 n 個費氏數是 $dp[n]$ 。)

C++ Implementation (Tabulation - 列表法):

```
#include <vector>
#include <iostream>

long long fib_tabulation(int n) {
    if (n <= 0) return 0;
    if (n == 1) return 1;

    std::vector<long long> dp(n + 1);
    dp[0] = 0; // F_0
    dp[1] = 1; // F_1

    for (int i = 2; i <= n; ++i) {
```

```

        dp[i] = dp[i-1] + dp[i-2];
    }
    return dp[n];
}

```

C++ Implementation (Memoization - 記憶化遞迴):

C++

```

#include <vector>
#include <iostream>
#include <map> // Or use a vector if n is reasonably small

// Using a map for memoization table for potentially sparse calls or larger n
// flexibility
// 使用 map 作為記憶化表格，適用於可能的稀疏呼叫或較大 n 值的彈性
std::map<int, long long> memo;

long long fib_memoization(int n) {
    if (n <= 0) return 0;
    if (n == 1) return 1;

    // Check if already computed
    // 檢查是否已經計算過
    if (memo.count(n)) {
        return memo[n];
    }

    // Compute and store
    // 計算並儲存結果
    memo[n] = fib_memoization(n - 1) + fib_memoization(n - 2);
    return memo[n];
}

```

Problem 1.1 fib_memoization(7) $\vdash$ fib_memoization(6) | $\vdash$ fib_memoization(5) | | $\vdash$ fib_memoization(4) | | | $\vdash$ fib_memoization(3) | | | | $\vdash$ fib_memoization(2) | | | | | $\vdash$ fib_memoization(1) = 1 | | | | | $\vdash$ fib_memoization(0) = 0 | | | | $\vdash$ memo[2] = 1 | | | $\vdash$ memo[3] = 2 | | | $\vdash$ memo[4] = 3 | | $\vdash$ memo[5] = 5 | $\vdash$ memo[6] = 8 $\Rightarrow$ memo[7] = 13

呼叫順序	儲存的子問題	memo 內容	1	memo[2] = 1	{2: 1}	2	memo[3] = 2	{2: 1, 3: 2}	3	memo[4] = 3	{2: 1, 3: 2, 4: 3}	4	memo[5] = 5	{2: 1, 3: 2, 4: 3, 5: 5}	5	memo[6] = 8	{2: 1, 3: 2, 4: 3, 5: 5, 6: 8}	6	memo[7] = 13	{2: 1, 3: 2, 4: 3, 5: 5, 6: 8, 7: 13}
------	--------	---------	---	-------------	--------	---	-------------	--------------	---	-------------	--------------------	---	-------------	--------------------------	---	-------------	--------------------------------	---	--------------	---------------------------------------

三種方法的複雜度比較表	方法	時間複雜度	空間複雜度	Naive Recursive	$O(2^n)$
$O(n)$ (遞迴深度)	Tabulation	$O(n)$	$O(n)$	Memoization	
$O(n)$ (每個計算一次)	$O(n)$ (儲存表)				

Problem 1.2 C++

```

#include <iostream>
#include <vector>

```

```

#include <map> // For memoization approach, if chosen

// ##### STUDENT'S IMPLEMENTATION AREA START #####
// ##### 學生的實作區域 START #####
long long count_ways_to_climb(int n) {
    if (n < 0) return 0; // 不可負階，視為 0 種方法
    std::vector<long long> dp(n + 1, 0);
    dp[0] = 1; // 到達第 0 階 (尚未開始爬) 有 1 種方法
    for (int i = 1; i <= n; ++i) {
        if (i >= 1) dp[i] += dp[i - 1];
        if (i >= 2) dp[i] += dp[i - 2];
        if (i >= 3) dp[i] += dp[i - 3];
    }
    return dp[n];
}
// ##### STUDENT'S IMPLEMENTATION AREA END #####
// ##### 學生的實作區域 END #####

int main_fib_climb() { // Renamed to avoid conflict if other mains are present
    using namespace std;
    cout << "--- Fibonacci / Climbing Stairs ---" << endl;
    // Test Problem 1.1 examples here (manual trace for memoization)
    // You can add calls to fib_tabulation and fib_memoization for testing.
    // 例如:
    // cout << "fib_tabulation(7): " << fib_tabulation(7) << endl;
    // memo.clear(); // Clear memo before each new independent test of
    fib_memoization
    // cout << "fib_memoization(7): " << fib_memoization(7) << endl;

    cout << "\n--- Climbing Stairs (1, 2, or 3 steps) ---" << endl;
    cout << "Ways to climb 0 stairs: " << count_ways_to_climb(0) << endl; //
Expected: 1
    cout << "Ways to climb 1 stair: " << count_ways_to_climb(1) << endl; //
Expected: 1
    cout << "Ways to climb 2 stairs: " << count_ways_to_climb(2) << endl; //
Expected: 2
    cout << "Ways to climb 3 stairs: " << count_ways_to_climb(3) << endl; //
Expected: 4
    cout << "Ways to climb 4 stairs: " << count_ways_to_climb(4) << endl; //
Expected: 1+2+4 = 7
    cout << "Ways to climb 5 stairs: " << count_ways_to_climb(5) << endl; //
Expected: 2+4+7 = 13
    return 0;
}

```

2. Longest Increasing Subsequence (LIS) (最長遞增子序列) C++ Implementation (Tabulation - 列表法):

C++

```

#include <vector>
#include <algorithm> // For std::max_element and std::max

int length_of_lis_tabulation(const std::vector<int>& nums) {
    if (nums.empty()) {
        return 0;
    }
    int n = nums.size();
    std::vector<int> dp(n, 1); // Initialize dp[i] = 1 for all i // 初始化所有 dp[i] = 1

    for (int i = 1; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            if (nums[j] < nums[i]) {
                dp[i] = std::max(dp[i], dp[j] + 1);
            }
        }
    }

    // The LIS length is the maximum value in the dp array
    // LIS 長度是 dp 陣列中的最大值
    int max_len = 0;
    if (n > 0) { // Ensure dp is not empty before accessing
        max_len = dp[0];
        for (int i = 1; i < n; ++i) {
            if (dp[i] > max_len) {
                max_len = dp[i];
            }
        }
    }
    // Or simply: if (n > 0) max_len = *std::max_element(dp.begin(), dp.end()); else
    max_len = 0;
    return max_len;
}

```

Problem 2.1 Step-by-step dp array after each outer loop (i):

$i = 0$: $dp = [1, , ,]$ → 初始設為 1
 $i = 1$: $dp = [1, 2, ,]$ → 因為 $10 > 3$ → $dp[1] = dp[0] + 1$
 $i = 2$: $dp = [1, 2, 1, ,]$ → $2 < 3, 2 < 10$ → 無更新
 $i = 3$: $dp = [1, 2, 1, 1,]$ → $1 < \text{others}$ → 無更新
 $i = 4$: $dp = [1, 2, 1, 1, 3]$ → $20 > 3, 10$ → 取 $dp[1] + 1 = 3$

Final LIS Length: 3

Reconstruction approach (重建 LIS 思路):

1. 在計算 $dp[i]$ 時，另外用一個 $prev[i]$ 陣列紀錄當前 LIS 最長時是從哪個 j 轉移過來的。
2. 找出 dp 中最大的那個 index。
3. 從這個 index 開始，利用 $prev[]$ 倒回去追蹤整條 LIS 路徑，組成真正的序列。

Problem 2.2 C++

```
#include <iostream>
#include <vector>
#include <algorithm> // For std::max or std::max_element

// ##### STUDENT'S IMPLEMENTATION AREA START #####
// ##### 學生的實作區域 START #####
int longest_increasing_subsequence(const std::vector<int>& nums) {
    int n = nums.size();
    if (n == 0) return 0;
    std::vector<int> dp(n, 1); // 每個元素先假設自己是長度為 1 的序列
    int max_len = 1;

    for (int i = 1; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            if (nums[j] < nums[i]) {
                dp[i] = std::max(dp[i], dp[j] + 1);
            }
        }
        max_len = std::max(max_len, dp[i]);
    }

    return max_len;
}

// ##### STUDENT'S IMPLEMENTATION AREA END #####
// ##### 學生的實作區域 END #####

int main_lis() { // Renamed to avoid conflict
    using namespace std;
    cout << "\n--- Longest Increasing Subsequence ---" << endl;

    vector<int> nums1 = {10, 9, 2, 5, 3, 7, 101, 18};
    cout << "LIS length for {10, 9, 2, 5, 3, 7, 101, 18}: "
         << longest_increasing_subsequence(nums1) << endl; // Expected: 4

    vector<int> nums2 = {0, 1, 0, 3, 2, 3};
    cout << "LIS length for {0, 1, 0, 3, 2, 3}: "
         << longest_increasing_subsequence(nums2) << endl; // Expected: 4

    vector<int> nums3 = {7, 7, 7, 7, 7};
    cout << "LIS length for {7, 7, 7, 7, 7}: "
         << longest_increasing_subsequence(nums3) << endl; // Expected: 1

    vector<int> nums4 = {3, 10, 2, 1, 20};
    cout << "LIS length for {3, 10, 2, 1, 20}: "
         << longest_increasing_subsequence(nums4) << endl; // Expected: 3

    vector<int> nums5 = {};
    cout << "LIS length for {}: "
         << longest_increasing_subsequence(nums5) << endl; // Expected: 0
    return 0;
}
```